

TYPESCRIPT



C'EST QUOI TYPESCRIPT ?

- ▶ **TypeScript** est un langage de programmation libre développé par Microsoft qui a pour but d'améliorer et de sécuriser la production de code JavaScript.
- ▶ **Typescript** est transcompilé en **Javascript**
 - ▶ Un code Javascript est un code Typescript, mais un code Typescript n'est pas un code Javascript
- ▶ Configuration TS, permettant de donner une limitation au langage



RÔLE DU *TSCONFIG.JSON*

- ▶ En Typescript vous devez inclure un fichier « **tsconfig.json** », il sert à indiquer le comportement du compilateur **tsc** ainsi que des règles de vérification de code
 - ▶ **exclude** : les éléments à exclure de la compilation
 - ▶ **sourceMap** : ajoute un fichier .js.map qui établit le lien entre le code Javascript généré et le code Typescript
 - ▶ **downlevelIteration** : améliore la transpilation des itérations modernes lorsque la cible JavaScript est ancienne
 - ▶ **experimentalDecorators** : autorise la syntaxe des décorateurs proposée par TypeScript
 - ▶ **module** : format de modules à générer dans le JavaScript
 - ▶ **target** : version minimale de JavaScript générée par TypeScript
 - ▶ **lib** : API JavaScript et navigateur que TypeScript considère disponibles lors de la vérification de type

```
{
  "exclude": [
    "node_modules"
  ],
  "compilerOptions": {
    "sourceMap": true,
    "downlevelIteration": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true,
    "module": "esnext",
    "moduleResolution": "node",
    "target": "es2015",
    "typeRoots": [
      "node_modules/@types"
    ],
    "lib": [
      "es2018",
      "es2019",
      "dom",
      "dom.iterable",
      "es2015.proxy"
    ],
  },
}
```



LES TYPES

- ▶ Une différence forte entre **Typescript** et **Javascript**, est le **typage des variables**, retour de fonction, propriétés d'objets etc
- ▶ Types primitifs : **string**, **number**, **boolean**, **null**, **undefined**
- ▶ Types de tableaux : **string[]**, **Array<string>**
- ▶ Types particuliers : **any**, **void**



LES TYPES DU DOM

- ▶ Tous les éléments du DOM extends « **Element** »
- ▶ On retrouve :
 - ▶ **HTMLElement** : tous les éléments HTML
 - ▶ **HTMLDivElement** : div
 - ▶ **HTMLButtonElement** : button
 - ▶ **HTMLInputElement** : input
 - ▶ **HTMLTableElement** : table
 - ▶ **HTMLLinkElement** : a



UTILISATION DES TYPES

- ▶ Typing a variable :
 - ▶ `const var: string = "Oui";`
 - ▶ `let test: Array<number> = [5, 2, 35];`
- ▶ Typing a function :
 - ▶ `function aFunction(param: string): void`
 - ▶ `function aFunction(param: number, paramTwo: string): string`
- ▶ On peut aussi représenter un retour multi-type : « `string | null` » ou encore « `string | number | null` »



LE “AS”

- En **Typescript** il existe le mot-clé « **as** » qui permet de typer le retour de certaines fonctions possédant un type trop générique, comme « **getAttribute** ». Le retour de cette fonction est de type « **string | null** », le plus pratique est de vérifier si la valeur n’est pas **falsy**.
- Il peut arriver que l’on soit sûr que la valeur soit forcément un « **string** », pour cela on peut s’assurer que la valeur soit un string avec le « **as** » :

```
div.getAttribute('data-name') as string;
```



OBJET ET CLASSE

- ▶ Au même titre qu'une variable on ajoute les « : » devant le type de l'attribut
- ▶ Déclarer « **private readonly element: HTMLElement** » effectue :
 - ▶ La déclaration d'un attribut privé
 - ▶ Non-modifiable (readonly)
 - ▶ De type HTMLElement

```
class ClickInput {  
  
    private readonly url: string;  
    private form: HTMLElement;  
  
    constructor(  
        private readonly element: HTMLElement,  
        private dataAttr: string = 'data-click-input'  
    )  
    {  
    }  
}
```




HÉRITAGE

- ▶ L'héritage se fait via le mot clé « **extends** »
 - ▶ **Square** est la classe enfant
 - ▶ **Shape** est la classe parente
- ▶ Le mot clé « **super** » rappelle les fonctions de la classe parente
 - ▶ Ici elle rappelle le constructeur

```
class Square extends Shape {  
  
    constructor()  
    {  
        super();  
    }  
  
}
```



CLASSE ABSTRAITE

- ▶ Une classe **abstraite** se déclare avec le mot clé « **abstract** »
- ▶ On déclare les attributs en « **protected** », si on peut les réutiliser dans les classes enfants
- ▶ Une classe abstraite peut avoir des **fonctions abstraites**, c'est-à-dire qu'elles doivent être implémentées dans la classe enfant
- ▶ Une classe abstraite ne s'instancie pas

```
abstract class Shape {  
  
    protected name: string;  
  
    abstract function test(): void;  
  
}
```



INTERFACE

- Une interface permet de déclarer des comportements à implémenter
- Ici on déclare une interface ayant une fonction « **getUrl** », toute classe implémentant cette interface doit donner une implémentation pour la méthode **getUrl**
- Une interface Typescript peut aussi contenir des attributs !

```
interface IGetUrl {  
  
    public function getUrl(): string;  
  
}
```



INTERFACE

► Elle peut aussi représenter les champs d'un JSON, on ne déclare que les attributs présents dans celui-ci

```
{  
  "id": 132,  
  "name": "ditto",  
  "atk": 40  
}
```

► L'intérêt est de pouvoir typer les retours de fonction « **fetch** » dans notre application

```
interface IPokemon {  
  id: number,  
  name: string  
}
```



GÉNÉRIQUE

- **Typescript** inclus la possibilité du typage générique : « **<T>** »
- Dans l'exemple suivant, cela implique que si on implémente l'interface « **Pagination** » on doit préciser quel type sera géré par cette interface
- Dans l'exemple ci-joint, la fonction « **getItems** » renvoie un objet « **Pagination** » avec « **Item** » entre chevrons. Cela montre que « **T** » dans l'interface prendra le type « **Item** »

```
interface Pagination<T> {  
  totalItems: number,  
  items: Array<T>,  
  page: number,  
  next: string | null,  
  previous: string | null  
}
```

```
function getItems(): Pagination<Item> {  
  
}
```



MODULE

- ▶ Un module évite que tous les fichiers de l'application partagent accidentellement les mêmes variables et responsabilités
- ▶ C'est un équivalent d'un **private** au niveau du fichier, les fonctions, types (classes ou interfaces) et variables sont exclusivement visible dans le fichier courant
- ▶ On choisit ce que l'on veut exporter via le mot-clé « **export** »
- ▶ Les fichiers choisissent ce qu'ils veulent importer via le mot clé « **import ... from** »



MODULE : EXEMPLE

► Prenons cet exemple, on définit une fonction « **calculerTva** » qui a le mot-clé « **export** », elle est donc accessible dans les autres fichiers, qui choisiront de l'importer

► On utilise le mot clé « **import** » pour utiliser la fonction « **calculerTva** » dans mon fichier courant. « **from** » indique le fichier où récupérer la fonction

```
const TVA = 0.2;
```

```
export function calculerTva(prixHT: number): number {  
    return prixHT * TVA;  
}
```

```
import { calculerTva } from "../calcul-panier";
```

```
const prixHT = 100;  
const montantTva = calculerTva(prixHT);
```



MODULE : EXEMPLE

► Si le fichier possède un export « **par défaut** », utilisation du mot-clé « **default** », alors on n'a pas besoin de mettre d'accolades dans l'import.

```
export default class ApiClient {  
  async get<T>(url: string): Promise<T> {  
    const response = await fetch(url);  
    return response.json;  
  }  
}
```

```
import ApiClient from "./api-client";
```